Vocabulary of Flaky Tests in Javascript

Rafael Rampim Soratto
Universidade Tecnológica Federal do Paraná
Programa de Pós-Graduação em Ciência da Computação
Campo Mourão, Paraná, Brasil
soratto@alunos.utfpr.edu.br

Marco Aurélio Graciotto Silva
Universidade Tecnológica Federal do Paraná
Programa de Pós-Graduação em Ciência da Computação
Campo Mourão, Paraná, Brasil
magsilva@utfpr.edu.br

ABSTRACT

Context: Regression testing is a software verification and validation activity in modern software engineering. In this activity, tests can fail without any implementation change, characterizing a flaky test. Flaky tests may delay the release of the software and reduce testing confidence. One way to identify flaky tests is by re-running the tests, but this has a high computational cost. An alternative to re-execution is the static analysis of the code of the test cases, identifying patterns related to flaky tests. **Objective:** The objective of this work was to identify flaky tests in Javascript applications by analyzing the source code of the test cases, without executing them. **Method:** A dataset was built with flaky test cases extracted from open source software hosted on GitHub and implemented in Javascript. Then, a classification model and a flakiness vocabulary were created, considering the source code of flaky tests in the Javascript language. **Results:** We observed good results during the execution of most classifiers using the training and validation sets, with the best result being the logistic regression algorithm. However, when classifying the test set, the performance was not good, with the best results being the linear discriminant analysis. We obtained a vocabulary related to instability with words associated with asynchronous behavior (*then*, *await*, *return*) and related to UI (*layout*, *gd*, *plot*, *click*). **Conclusions:** This work presents relevant results toward a more efficient identification of flaky tests in projects that use Javascript. Further studies are required to consolidate a reliable classification of tests regarding flakiness using the vocabulary approach.

ACM Reference Format:
Rafael Rampim Soratto and Marco Aurélio Graciotto Silva. 2023. Vocabulary of Flaky Tests in Javascript. In *XXII Brazilian Symposium on Software Quality (SBQS '23), November 07–10, 2023, Brasília, Brazil*. ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3629479.3629487

1 INTRODUÇÃO

O teste de regressão é uma atividade importante para verificação e validação de sistemas presente na engenharia de software moderna [36]. Ele é responsável por garantir que as funcionalidades do software não sejam danificadas com futuras atualizações, mantendo o comportamento esperado na especificação do produto.

Assim como outras funcionalidades do software, o teste precisa ser implementado em código-fonte para ser executado. Espera-se que o resultado do teste seja mantido se o código-fonte não for alterado. Porém, ele pode variar (*e.g.*, de falso para verdadeiro e vice-versa) em diferentes execuções sem alterações de implementação. Este evento caracteriza o teste como *flaky test* ou teste instável.

Este problema intermitente é relatado principalmente em empresas sólidas no ramo de tecnologia, tais como Microsoft [24, 27], Spotify [39], Google [34, 35] e Facebook [20]. Isto ocorre pois elas possuem uma grande quantidade de testes que são executados inúmeras vezes durante o processo de entrega do software.

A causa raiz da instabilidade em testes varia, aumentando a complexidade de sua identificação e correção. Portanto, surgiram ferramentas focadas em reduzir os recursos exigidos dos desenvolvedores e dos sistemas para identificar e corrigir *flaky tests* [6, 6, 14, 15, 21, 31, 32, 32]. Essa automatização permite economizar recursos dos desenvolvedores e, se a abordagem for leve, também permite economizar recursos do sistema.

A maneira mais comum para se identificar *flaky tests* é a reexecução, geralmente com a variação das condições do teste. No entanto, muito recurso é gasto para identificar instabilidades nesta estratégia devido à quantidade de reexecuções e ao custo associado a cada uma delas [28, 34]. Como alternativa, podem ser utilizadas abordagens que não dependem da reexecução, empregando técnicas de aprendizado de máquina que exploram regras de associação e classificadores para a predição de instabilidade [8, 24].

O custo alto para identificação de instabilidades em testes motiva o desenvolvimento de abordagens de predição como a abordagem de vocabulário. A abordagem de vocabulário une as áreas de engenharia de software e inteligência artificial para sugerir ao desenvolvedor que o código possui a chance de falhar sem a necessidade de múltiplas execuções de testes, facilitando seu emprego em ambientes de produção de software como integração e implantações contínuas, sugestão de código e analisadores estáticos de qualidade de código.

O objetivo deste trabalho foi identificar instabilidades em aplicações Javascript, utilizando a abordagem de vocabulário. Desta forma, as metas abrangeram: criação de um conjunto de dados sobre *flaky tests* para Javascript; a criação e avaliação de modelos de classificação considerando o vocabulário dos *flaky tests* em Javascript.

Este trabalho é organizado da seguinte forma. Na Seção 2 são apresentados os conceitos e trabalhos relacionados a *flaky tests* e técnicas para sua identificação. Na Seção 3 é apresentada a abordagem deste trabalho, abrangendo técnicas de processamento de texto quanto a código Javascript, construção de conjuntos de dados, modelos de classificação e criação de vocabulário de *flaky tests* para Javascript. Tais conjuntos de dados são apresentados na Seção 4. Os resultados quanto da avaliação da qualidade dos classificadores são apresentados na Seção 5. Na Seção 6 comparamos os nossos resultados com os trabalhos relacionados. Ameaças à validade são

Publication rights licensed to ACM. ACM acknowledges that this contribution was authored or co-authored by an employee, contractor or affiliate of a national government. As such, the Government retains a nonexclusive, royalty-free right to publish or reproduce this article, or to allow others to do so, for Government purposes only.
SBQS '23, November 07–10, 2023, Brasília, Brazil
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0786-5/23/11…$15.00
https://doi.org/10.1145/3629479.3629487

discutidas na Seção 7. Finalmente, conclusões deste estudo e delineamento de trabalhos futuros estão descritos na Seção 8.

2 REFERENCIAL TEÓRICO

Nesta seção é definido o conceito de um teste instável, apresentando técnicas de identificação e conjuntos de dados sobre *flaky tests*.

2.1 Caso de teste instável (flaky test)

O *flaky test* representa uma variação no resultado do teste em diferentes execuções e sem variação de implementação. Este comportamento não determinístico remove parte da confiança nos testes e geralmente é resultado de um cenário não previsto para a execução. A grande diferença entre uma falha comum e a instável é: a falha comum ocorre após alguma alteração no software, quebrando o valor do teste até que ele não seja corrigido; enquanto a instabilidade quebra o valor somente algumas vezes durante uma sequência de testes, sem nenhuma alteração no código fonte [15].

Na Listagem 1 é apresentado um *flaky test* do projeto Moleculer[1], cuja causa refere-se à utilização de data e hora. Um objeto é criado e um de seus atributos é a data atual. Em sequência, é realizada a serialização do objeto e uma asserção sobre o tamanho do objeto serializado. No entanto, a depender da hora em que o teste é executado, é inserido um sufixo com o fuso horário no atributo de data serializado, alterando o tamanho do objeto. Consequentemente, ocorre a falha inesperada do caso de teste, ou seja, um *flaky test*.

Listagem de código 1: Código a ser tokenizado.

```
1  it("should serialize the event packet", () => {
2    const now = new Date();
3    const obj = {
4      ver: "4",
5      sender: "node-100",
6        id: "8b3c7371-7f0a-4aa2-b734-70ede29e1bbb",
7        event: "user.created",
8        data: {
9            a: 5,
10           b: "Test",
11           c: now
12       },
13       broadcast: true,
14       meta: {},
15       level: 1,
16       needAck: false
17   };
18   const s = serializer.serialize(cloneDeep(obj),
       P.PACKET_EVENT);
19   expect(s.length).toBe(150);
20   const res = serializer.deserialize(s,
       P.PACKET_EVENT);
21   expect(res).not.toBe(obj);
22   expect(res).toEqual(obj);
23 });
```

2.2 Técnicas para identificação

Existem diversas técnicas para prevenção, identificação, previsão e correção de *flaky tests* [6, 9, 20, 37, 40]. A identificação de *flaky tests* contribui tanto para indústria, permitindo a descoberta de instabilidades em casos de teste de aplicações em desenvolvimento e em produção, quanto para a academia, permitindo aos pesquisadores identificar futuras instabilidades e criar abordagens variadas para isto. Testes rotulados como instáveis podem ser úteis para prever futuras instabilidades utilizando a abordagem de reexecução e de reconhecimento de padrões [15, 32]. Neste trabalho, o foco reside na identificação de testes instáveis, buscando estabelecer uma técnica estática e de baixo custo computacional.

2.2.1 Identificação por reexecução. Uma técnica muito utilizada é a reexecução. Ela consiste na repetição da execução do conjunto de casos de teste de uma aplicação. Dessa forma, essa técnica é similar ao que ocorre nos testes de regressão, que inicialmente motivaram a investigação sobre testes instáveis. A reexecução permite o estudo sobre a correlação (temporal e espacial) em falhas de uma mesma suíte de testes. Por exemplo, vários testes podem depender de algum serviço de rede e quando um teste falha (se o serviço for desativado), todos os outros testes provavelmente também falharão [15].

No entanto, essa técnica de reexecução é pouco eficiente [15], exigindo uma quantidade elevada de execuções para aumentar a chance de identificar um caso de teste instável. Alguns estudos adotam o padrão de 10.000 execuções para estabelecer a classificação de um caso de teste. Logo, essa estratégia não é adequada para estabelecer a instabilidade de casos de teste em um ambiente de produção, motivando a investigação por técnicas mais eficientes.

Os testes de sistemas são um grande desafio pois possuem comunicações complexas e assíncronas, simultaneidade entre os clientes-servidores e um padrão dos recursos empregados. É difícil manter a condição destes testes pois eles podem ser executados de formas variadas, como por exemplo: com diversos fatores ambientais, diferentes disponibilidades de rede e memória, e até mesmo com variação da resolução da tela. Essa condição variável pode resultar em instabilidades nos testes. O trabalho "*FlakyLoc*" utiliza a reexecução em diferentes configurações de teste utilizando uma abordagem combinatória para localizar a instabilidade [37].

Para diminuir o custo deste processo de identificação, surgem técnicas estáticas, que identificam instabilidades utilizando informações estáticas presentes em casos de testes ou obtidas a partir de poucas reexecuções.

2.2.2 Identificação a partir de reconhecimento de padrões. Abordagens estáticas foram propostas para reduzir o custo de identificação de instabilidade geralmente utilizando técnicas de aprendizagem de máquina [24, 26, 40]. A abordagem chamada vocabulário instável visa encontrar identificadores textuais que estão fortemente relacionados com *flaky tests*. Nesta técnica é necessário extrair identificadores de casos de testes e verificar quais identificadores estão fortemente associados à instabilidade. Quando os identificadores ocorrem recorrentemente em testes instáveis, pode-se usar modelos de classificação para se construir um vocabulário instável [40].

No primeiro trabalho sobre o vocabulário instável, Pinto et al. [40] utilizaram o conjunto de dados construído com base em 24 projetos analisados pelo DeFlaker [6]. Vocabulários necessitam que os casos de testes sejam transformados em identificadores textuais ou características para utilização do método de predição. Para se utilizar dados textuais podem ser utilizadas técnicas de processamento de texto, incluindo: criação de identificadores de palavras (*tokens*), separação de palavras em termos que utilizam *CamelCase* ou similares, lematização, remoção de palavras irrelevantes [40]. Nesse trabalho, propõe-se uma medida de precisão diferente da acurácia convencional, pois desconsidera a predição de "não instáveis", removendo-se os "verdadeiros negativos" e "falsos negativos".

[1] https://github.com/moleculerjs/moleculer

O foco é dado na pontuação F1 (a média harmônica de precisão e *recall*), pois há um maior interesse em prever corretamente *flaky tests* ao invés de não *flakies*.

Como resultado, Pinto et al. [40] descreveram alguns pontos interessantes. Primeiro, foram encontrados seis projetos com testes instáveis e um total de 86 testes instáveis neles. Um total de 55% desses *flaky tests* falharam apenas uma vez, o que significa que o limite de 100 execuções pode ter limitado a observação de testes instáveis (ou seja, é provável que possamos encontrar mais *flaky tests* com mais execuções). Quanto aos algoritmos de aprendizagem de máquina utilizados para criar os modelos de classificação, o Random Forest teve a melhor precisão (0,99), enquanto Support Vector Machine teve um desempenho ligeiramente superior ao Random Forest em termos de *recall* (0,92 vs 0,91). Por fim, em relação ao vocabulário de *flaky tests*, foram identificadas palavras como *job*, *table*, *action*, muitas das quais estão associados à execução de tarefas remotamente ou usando uma fila de eventos [40].

Estudos subsequentes replicaram a abordagem de vocabulário instável, considerando mais modelos de classificação e outras linguagens de programação. De modo geral, foram apresentados resultados positivos, indicando que a utilização de vocabulários é replicável e extensível. Porém o nível de generalização do vocabulário pode variar em diferentes contextos e aplicações, tornando o vocabulário muito dependente do contexto [4, 6, 10, 15, 40].

Além dos termos dos casos de teste, é possível considerar outras características dos casos de teste. *Test smell* é um conjunto de sinais de más escolhas na codificação de teste [13]. Estes sinais podem alertar ou até identificar possíveis falhas no teste, motivando diversos estudos relacionados [5, 13, 45]. Em geral, eles são introduzidos durante a criação do teste e duram por muito tempo, sobrevivendo até milhares de *commits*. Por estar relacionado com testes e com falhas (intermitentes ou não), este assunto foi objeto de estudo em conjunto com *flaky tests* [9]. A ideia principal da abordagem é que a instabilidade dos casos de teste é uma das manifestações do efeito negativo de *test smells* quanto à qualidade dos casos de teste.

A ferramenta chamada *FlakyFlagger* propõe a predição de instabilidade com base em recursos comportamentais semelhantes [4]. Outro sistema chamado *Flast* adota uma abordagem estática para predição de instabilidade. Nesta processo, ele realiza etapas de modelagem de espaço vetorial, pesquisa de similaridade, redução de dimensionalidade e classificação de *flaky tests* [46].

2.3 Conjuntos de dados empíricos sobre *flaky tests*

Estudos sobre *flaky tests* necessitam de relatos e ocorrências empíricas sobre instabilidade para que sejam projetadas soluções viáveis para o problema de instabilidade em testes em um cenário real. Com estes dados empíricos são projetadas abordagens para tratar a instabilidade. Desta forma podemos validar as abordagens de identificação de instabilidades em um cenário de produção de software real. Existem estudos que utilizam conjuntos de dados empíricos sobre casos de testes implementados na linguagem Java para servir de base para abordagens de identificação de instabilidade [6, 10, 40]. No entanto, existem poucos trabalhos que fornecem um conjunto de dados sobre *flaky tests* em Javascript [22, 43].

No conjunto de dados do Romano et al. [43], apresenta-se uma abordagem de coleta de instabilidades em testes que identifica a palavra *flaky* e *flakiness* em alterações de código (*commit*), solicitações de alteração (*pull request*) e relatos de problemas (*issues*) em projetos abertos (do GitHub) em Java e Javascript. O resultado desta coleta foi um total de 235 instâncias de instabilidades em testes automáticos em Javascript e Java. Cada instância possui um conjunto de atributos textuais que ajudam a identificar a natureza da instabilidade, utilizando as informações fornecidas pelo desenvolvedor que relatou a instabilidade e o seu contexto. Existem instâncias do conjunto de dados que contêm o link para os *commits* nos quais o autor utiliza termos relacionados a instabilidade em testes. Os atributos de cada instância do conjunto de dados são as seguintes: título, URL, causa raiz da instabilidade, categoria de causa raiz, código afetado (URL), ambiente de teste, como foi corrigido, categoria de correção, link do *pull request*, categoria de manifestação.

Hashemi et al. [22] também coletaram casos de teste instáveis em Javascript de projetos mantidos no GitHub. Os projetos foram selecionados pela relevância (número de estrelas e *forks*). Um total de 316.948 *commits* foram analisados, dos quais 735 estavam relacionados com *flaky tests*. Após um filtro, Hashemi et al. [22] chegaram no número de 452 commits considerados relevantes que foram classificados manualmente. Por fim, foi categorizada a causa raiz de instabilidade de um total de 358 commits.

Uma limitação dos trabalhos de Hashemi et al. [22] e Romano et al. [43] é que eles não fornecem claramente o código-fonte do caso de teste rotulado como instável. Isso impede a aplicação direta de abordagens como a de vocabulário.

3 MÉTODO DE PESQUISA

Este trabalho adotou a abordagem de vocabulário instável para identificar *flaky tests* em casos de teste escritos em Javascript, de forma similar a trabalhos anteriores que trataram de software escrito em Java [10, 40] e Python [19]. As questões de pesquisa abordadas foram:

> **QP1**: Com que qualidade podemos prever instabilidades em casos de teste escritos em Javascript?

> **QP2**: Quais identificadores presentes em códigos de testes estão fortemente associados à instabilidade?

Para responder a essas questões de pesquisa, estabelecemos um modelo de classificação com base nos termos presentes nos casos de teste. A mineração de dados, que consiste no processo de extrair conhecimento útil e previamente desconhecido de dados [17], foi empregada para identificar padrões representativos quanto a esses termos. Esse processo de mineração é organizado em etapas: conhecimento do domínio, pré-processamento dos dados, extração de padrões, pós-processamento e aplicação do conhecimento [42].

Tal processo foi utilizado no contexto da criação de um vocabulário de instabilidade para casos de teste em Javascript. Desta maneira, foi possível descobrir quais termos estão presentes em instabilidades de testes. Para realizar este processo foram definidas duas etapas, conforme mostrado na Fig. 1: representação dos casos de teste, considerando a frequência dos *tokens* extraídos do código dos testes do conjunto de dados; e criação e avaliação de modelos

de classificação de casos de teste quanto à estabilidade, definindo-se o vocabulário de testes instáveis.

Para tanto, este trabalho requer um conjunto de dados sobre *flaky tests* em Javascript. No entanto, são poucos os estudos neste tópico [22, 43] e nenhum deles estabeleceu um conjunto de dados que identifica os casos de teste com instabilidade. Assim, foi necessário criar esse conjunto, coletando casos de teste implementados em Javascript obtidos a partir de: (1) conjunto de testes estabelecidos empiricamente por Romano et al. [43], com casos de teste instáveis identificados de forma empírica (mas sem a identificação de casos de teste sem instabilidade); e (2) conjuntos de testes avaliados quanto a instabilidade com o emprego da técnica de reexecução. Uma vez estabelecido esse novo conjunto de dados, avaliamos a capacidade de predição de instabilidade com base nos identificadores presentes no código-fonte dos casos de teste Javascript.

Quanto ao conhecimento de domínio, foi necessário identificar como os casos de teste são especificados em software desenvolvido em Javascript. De forma similar aos trabalhos relacionados a Java e Python, os casos de teste automatizados estão vinculados a arcabouços para especificação de casos de teste, tais como Jest [16], Cypress [12] e Karma [25]. Tais arcabouços estabelecem uma forma padronizada para criação de casos de teste, com uso de funções como *describe* e *test* para especificar os casos de teste, além de conjuntos de funções auxiliares, como *expect* e *toBe*, para definir as assertivas. Os conjuntos de casos de teste, por sua vez, são organizados em arquivos localizados em algum diretório padronizado (*tests*) ou com um sufixo que facilite a categorização como casos de teste (*.test.js*). Além disso, a execução dos casos de teste é coordenada por programas para construção de aplicações Javascript, tal como NPM [38] e Yarn [33]. O foco neste artigo foi em projetos que utilizaram os arcabouços Jest e Karma e que podiam ser construídos com NPM.

O pré-processamento de dados consistiu na obtenção dos casos de teste de projetos Javascript e a extração dos termos de cada caso de teste, na forma de *tokens*. Diferentemente dos outros trabalhos, para a obtenção dos *tokens* foi utilizado o analisador sintático ESLint-TypeScript [23] ao invés de heurísticas, permitindo extraí-los com precisão. A partir desses *tokens*, foi criada uma representação vetorial de cada caso de teste, considerando todos os *tokens* do conjunto de dados, na forma do modelo espaço-vetorial (*bag of words*).

Os padrões foram extraídos considerando um conjunto de 8 algoritmos de aprendizagem de máquina de modo similar aos trabalhos relacionados: Linear Discriminant Analysis (LDA), Random Forest, Decision Tree, Naive Bayes, Sequential Minimal Optimization (SMO), K-Neighbor (Knn), Logistic Regression (LR) e Perceptron. Cada modelo foi treinado e validado com casos de teste identificados a partir do estudo do Romano et al. [43] e testado com o casos de teste classificados por reexecução. Para replicar a abordagem dos trabalhos anteriores, separamos 80% para treinamento e 20% para validação, de forma aleatória [10, 40]. O resultado final foi avaliado quanto à acurácia, precisão, *recall*, F1-score médio, AUC, MCC e o F1-score dos acertos (verdadeiros positivos e negativos).

No pós-processamento, foram analisados os termos mais relevantes e que possuem maior número de ocorrências em *flaky tests*. Esses termos foram identificados pelo ganho de informação.

Finalmente, quanto à aplicação do conhecimento, o classificador resultante pode ser utilizado em outros projetos Javascript, permitindo a classificação de casos de teste quanto à instabilidade.

Destes passos, as principais contribuições deste trabalho referem-se à criação de um conjunto de dados sobre testes instáveis em Javascript e ao vocabulário para classificação de casos de teste quanto à instabilidade. As próximas duas seções abordam esses tópicos, respectivamente.

4 CONJUNTO DE DADOS SOBRE TESTES INSTÁVEIS EM JAVASCRIPT

A identificação de casos de teste instáveis pode ser realizada analisando *commits* de desenvolvedores que relataram *flaky tests* [32] em projetos de software livre. A análise empírica de *commits* e seus arquivos alterados e mensagens sobre as alterações (*changelog*), complementada pela análise de solicitações de alterações (*pull requests*) e relatos de erro (*issues*), permite obter informações a instabilidade em testes. De modo geral, os trabalhos que adotam essa técnica restringem a análise a *commits* anotados pelos desenvolvedores com mensagens associadas a instabilidade [18, 22, 32, 43, 44]. Assim, essa abordagem é efetiva para identificar casos de teste instáveis. No entanto, ela possui uma limitação por não permitir classificar a instabilidade dos demais casos de teste dos softwares sob análise.

A segunda abordagem para estabelecer os conjuntos de dados é pela reexecução de todos os casos de teste dos projetos [7, 29, 30, 40]. Embora tenha um custo elevado de execução, essa técnica permite

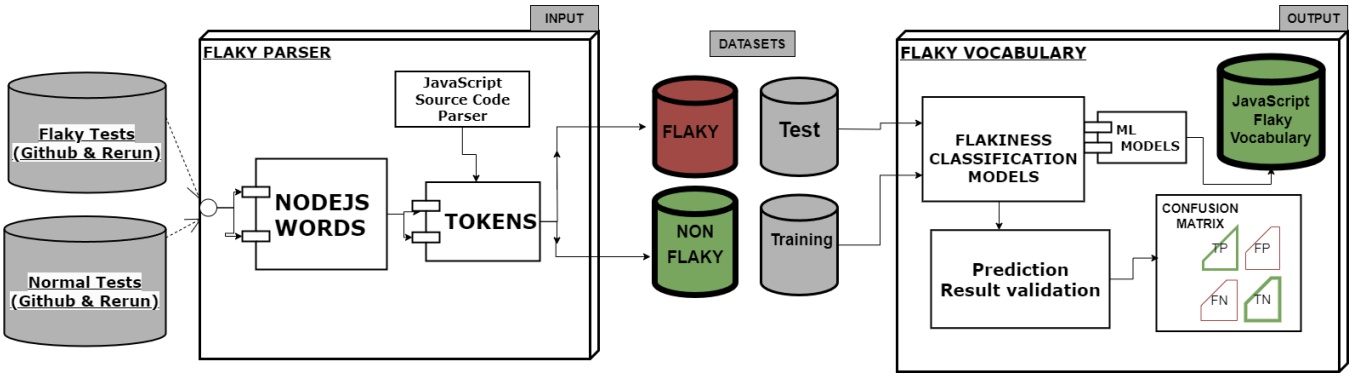


Figura 1: Processo para construção de um vocabulário de testes instáveis em Javascript.

classificar um caso de teste entre estável e instável (*flaky*), o que é necessário para a avaliação apropriada dos classificadores utilizados para identificar casos de teste instáveis.

Neste trabalho, adotamos a abordagem empírica de revisão de *commits* em projetos que relataram instabilidades no GitHub conforme observado por Romano et al. [43], com o objetivo de construir um conjunto de dados que consiste em testes instáveis. Também aplicamos a estratégia de reexecução em alguns projetos, estabelecendo a classificação de todos os casos de teste e identificar novos casos de teste instáveis. As ferramentas utilizadas a criação do conjunto de dados e o conjunto de dados propriamente ditos estão disponíveis no pacote de dados do projeto[2].

4.1 Formato do conjunto de dados

Conforme apresentado no Listagem 1, o código de um caso de teste está especificado em uma função, de acordo com o arcabouço utilizado para a criação e automatização de testes. No exemplo citado, o código compreende o conteúdo entre as linhas 2 e 22.

Para o conjunto de dados estabelecido nesse trabalho, o conteúdo de cada caso de teste foi representado pela frequência com que cada termo (*token*) ocorreu em seu código. Assim, o próximo passo foi transformar o código de teste em uma sequência de *tokens*. Considerando ainda o exemplo da Listagem 1, os *tokens* obtidos são aqueles da Listagem 2.

Listagem de código 2: *Tokens* do código do caso de teste de Listagem 1.

```
const, now, new, Date, const, obj, ver, 4, sender,
 node-100, id, event, user.created, data, a, 5, b,
 Test, c, now, broadcast, true, meta, level, 1,
needAck, false, const, s, serializer, serialize,
cloneDeep, obj, P, PACKET_EVENT, expect, s,
length, toBe, 150, const, res, serializer,
deserialize, s, P, PACKET_EVENT, expect, res, not,
 toBe, obj, expect, res, toEqual, obj
```

Conforme mostrado na Listagem 2, existem *tokens* que se repetem: *const*, *expect*, *now*, *obj*, *P*, *PACKET_EVENT*, *res*, *s*, *serializer*, *toBe*. Contabilizamos o número de vezes que cada *token* ocorre e, com isto, representamos cada caso de teste com essa informação.

Finalmente, registramos o conjunto de *tokens* extraído de todos os casos de teste. De fato, durante a computação da classificação, representamos cada caso de teste como um vetor com todos os *tokens* observados no conjunto de dados, usando os valores de frequência identificados e, na ausência de um *token* da coleção de dados em um caso de teste específico, a frequência daquele *token* para aquele caso de teste é zero. Assim, ao final, podemos representar o conjunto de dados em um modelo espaço-vetorial de alta dimensionalidade e esparso.

4.2 Conjunto de dados empírico

A identificação de casos de teste instáveis de forma empírica usualmente recupera dados de repositórios de projetos de código aberto, escolhidos em ambientes sociais de desenvolvimento como o GitHub [41]. Para facilitar a comparação dos resultados deste trabalho com os trabalhos relacionados, optou-se por escolher um conjunto de projetos de software livre analisados no trabalho de Romano et al. [43]. Ele contempla 36 projetos em Javascript relacionados a arcabouços para construção de interfaces gráficas, contendo 235 *commits* referentes a *flaky tests*.

Uma limitação do conjunto de dados de Romano et al. [43] é que ele identifica os arquivos relacionados com a instabilidade do teste, mas não o caso de teste específico. Dessa forma, é necessário extrair o caso de teste estritamente associado à instabilidade. Isso foi feito pela análise do *commit*, extraindo-se de forma automática os casos de teste dos arquivos alterados com o auxílio de um analisador sintático de Javascript e se analisando manualmente cada caso de teste extraído, verificando-se a pertinência do caso de teste para a instabilidade em questão. Após realizado esse processo, foram identificados 144 casos de teste instáveis e mais de 23 mil casos de teste com estabilidade desconhecida (supostamente não *flakies*) em 32 dos projetos, conforme apresentado na Tabela 1. Seguindo a abordagem apresentada no trabalho de Pinto et al. [40], reduzimos a quantidade de testes não instáveis para se aproximar da quantidade de testes instáveis, selecionando aleatoriamente 724 casos de teste para utilizar nos modelos de classificação.

4.3 Conjunto de dados por reexecução

O segundo conjunto de casos de teste foi obtido pela reexecução de projetos Javascript. O conjunto de casos de teste de cada projeto foi reexecutado 1.000 vezes. Buscou-se selecionar projetos que tinham sido analisados anteriormente de forma empírica, limitando-se a quantidade de projetos pelo tempo disponível para execução dos estudos. Além disso, foram analisados outros dois projetos: Moleculer[3] e Xtate[4]. Eles foram selecionados pois fazem parte do mesmo domínio dos projetos analisados anteriormente (interface gráfica ou que utilizam funções assíncronas para tratamento de dados na Web) e possuem relevância e visibilidade (em estrelas no GitHub) semelhantes aos projetos selecionados por [43]. Na Tabela 2 são apresentados os resultados da reexecução, marcando-se em negrito os projetos em comum com o conjunto de dados empírico.

Ao todo, foram identificados 15 casos de teste instáveis e 4.392 casos de teste estáveis. Cabe destacar que a quantidade de casos de teste para um mesmo projeto das Tabelas 1 e 2 é diferente porque as versões dos projetos analisados eram diferentes: no caso do estudo empírico, a versão correspondo ao *commit* analisado empiricamente enquanto que, para a reexecução, foi analisada a última versão do software em setembro de 2022.

5 RESULTADOS DA ABORDAGEM

Estabelecidos os conjuntos de dados, é possível criar os modelos de classificação e estabelecer o vocabulário para os testes instáveis em Javascript, para então responder as duas questões de pesquisa, conforme apresentado a seguir.

5.1 Classificação de casos de teste em Javascript quanto à instabilidade

A criação de modelos para classificação de testes instáveis considerou o conjunto de dados empírico para treinamento e validação (primeiro cenário) e o conjunto de dados obtido por reexecução

[2]https://github.com/sorattorafa/javascript-flaky-vocabulary

[3]https://moleculer.services/index.html

[4]https://github.com/statelyai/xstate

Tabela 1: Informações sobre o conjunto de dados obtido empiricamente.

Projeto	# flaky	# desconhecido
angular/angular	6	2.896
angular/components	10	6.859
apollographql/react-apollo	1	0
coralproject/talk	1	329
enzymejs/enzyme	1	0
gatsbyjs/gatsby	1	21
gotify/server	6	73
Hacker0x01/react-datepicker	3	0
HospitalRun/hospitalrun-frontend	1	150
influxdata/influxdb	22	0
JetBrains/ring-ui	2	1.214
jhipster/generator-jhipster	3	2.464
mobxjs/mobx	2	47
mui-org/material-ui	3	407
NativeScript/nativescript-angular	1	0
nfl/react-helmet	1	0
OfficeDev/office-ui-fabric-react	1	0
outline/outline	2	0
palantir/blueprint	3	31
pinterest/gestalt	4	582
plotly/plotly.js	32	102
preactjs/preact	17	970
react-cosmos/react-cosmos	1	47
rjsf-team/react-jsonschema-form	1	0
skbkontur/retail-ui	1	0
storybookjs/storybook	1	1.555
stream-labs/streamlabs-obs	2	0
styleguidist/react-styleguidist	1	374
thaliproject/postcardapp	5	0
twbs/bootstrap	1	763
uber/baseweb	7	593
vercel/next.js	1	3.871
Total: 32 projetos	**144 testes**	**23.294 testes**

Tabela 2: Informações sobre o conjunto de dados obtido por reexecução.

Projeto	# flaky	# não-flaky
mobxjs/mobx	**0**	**156**
moleculerjs/moleculer	4	6
react-cosmos/react-cosmos	**0**	**230**
statelyai/xstate	0	924
styleguidist/react-styleguidist	**0**	**510**
twbs/bootstrap	**0**	**783**
vercel/next.js	**11**	**1.665**
Total: 7 projetos	**15 testes**	**4.392 testes**

para testar o classificador obtido (segundo cenário). Conforme mencionado na Seção 3, os seguintes algoritmos de aprendizagem foram aplicados: Linear Discriminant Analysis (LDA), Random Forest, Decision Tree, Naive Bayes, Sequential Minimal Optimization (SMO), K-Neighbor (KNN), Logistic Regression (LR) e Perceptron. A implementação dos modelos de classificação foi realizada na linguagem Python e com a biblioteca Scikit-Learn [11]. Para definir um ambiente de classificação similar com os trabalhos relacionados [40], separamos o conjunto de dados contendo testes normais e instáveis em 20% para teste e 80% para treinamento.

Avaliamos o desempenho dos algoritmos de aprendizado de máquina em nosso conjunto de dados em termos de métricas comumente usadas na literatura: precisão, *recall*, *F1-score*, MCC e AUC. As métricas foram obtidas a partir da matriz de confusão de cada modelo. Nosso foco é na pontuação F1 (média harmônica de precisão e *recall*) pois há um maior interesse em prever corretamente *flaky tests* ao invés de não *flakies*, considerando o custo relacionado a resolução de eventual instabilidade [40].

No primeiro cenário utilizamos o mesmo conjunto para treino e teste dos modelos de classificação, contemplando 144 instâncias *flaky tests* e 724 casos de testes normais (com instabilidade desconhecida e, supostamente, estáveis) selecionados aleatoriamente dos 23.294 testes listados na Tabela 1. Com esse conjunto, treinamos e testamos os modelos de classificação, obtendo a matriz de confusão, conforme apresentado na Tabela 3. Os verdadeiros positivos (TP) são instâncias que são *flaky* e são rotuladas como tal. Já os falsos negativos (FN) são instâncias que são *flaky* e são rotuladas como desconhecidas (ou não flaky). Para um modelo ser considerado bom, em nosso cenário, esperamos que o número de TP seja alto e os FN e FP sejam baixos. Desta forma, o modelo aprende corretamente o que é ou não *flaky*, reduzindo o erro dos rótulos.

Na Tabela 3 são apresentados os resultados da matriz de confusão de cada algoritmo. Com base nessa matriz, obtivemos os resultados das classes *True* e *False* e, com base nisso, calculamos a média ponderada. Nos trabalhos anteriores apenas a média ponderada foi apresentada [10, 40]. Porém, por causa de nosso conjunto de dados desbalanceado, os resultados da média ponderada são demasiadamente positivos por causa dos (supostos) negativos (casos de teste estáveis). Portanto, para ter uma visualização mais fiel sobre a capacidade de cada algoritmo, apresentamos os resultados da classe *True*, o que revela a capacidade de cada modelo de encontrar *flaky tests*.

Falando sobre os resultados da média ponderada, o melhor resultado é o do KNN, apresentando *F1-score*, acurácia, AUC e *recall* de 0.98. Os modelos resultantes da utilização de árvores de decisão e regressão logística também tiveram bons resultados. De modo geral, os modelos tiveram desempenho satisfatório, com *recall* em nosso primeiro cenário variando entre 0.94 até 0.98. Já o F1-score apresentou resultados em um intervalo de 0.76 até 0.98. A coluna MCC representa uma predição perfeita quanto mais próxima do número 1. Apenas os algoritmos Naive Bayes e SVM apresentaram um valor abaixo do mínimo esperado (0.5).

Quanto ao resultado referente à classe *True*, no primeiro cenário possuímos bons resultados para identificar os exemplos positivos de instabilidade. O melhor resultado foi para o modelo de regressão logística, com F1-score de 0.82, *recall* de 0.72 e MCC de 0.82. O modelo resultante do KNN também possuiu um desempenho equiparável, embora com menor F1-score e Recall. É importante para nosso modelo a correta identificação de *flaky tests*, dada pela

Tabela 3: Resultados dos algoritmos de classificação para o conjunto de treinamento e validação.

Algoritmo	Matriz				Média ponderada dos resultados				Classe *True*			
	TN	FP	FN	TP	F1-Score	Accuracy	AUC	Recall	F1	AUC	MCC	Recall
Random Forest	**724**	**0**	22	15	0.97	0.97	**0.98**	0.97	0.58	0.98	0.63	0.4
Decision Tree	719	5	11	26	**0.98**	**0.98**	**0.98**	0.97	0.76	0.82	0.76	**0.83**
Naive Bayes	486	238	**5**	**32**	0.77	0.68	0.77	0.94	0.21	0.77	0.24	0.11
SMO	**724**	**0**	35	2	0.93	0.95	0.96	0.95	0.10	**0.96**	0.23	0.05
KNN	**724**	**0**	12	25	**0.98**	**0.98**	**0.98**	**0.98**	0.81	0.84	**0.82**	0.67
Logistic Regression	722	2	10	27	**0.98**	**0.98**	0.95	**0.98**	**0.82**	0.95	**0.82**	0.72
Perceptron	**724**	**0**	25	12	0.96	0.97	0.90	0.96	0.49	0.90	0.56	0.32
LDA	708	16	14	23	0.96	0.96	0.76	0.96	0.61	0.76	0.58	0.62

Tabela 4: Resultados dos algoritmos de classificação para o conjunto de teste.

Algoritmo	Matriz				Média ponderada dos resultados				Classe *True*			
	TN	FP	FN	TP	F1-Score	Accuracy	AUC	Recall	F1	AUC	MCC	Recall
Random Forest	4391	1	14	0	0.99	1.00	0.59	0.99	0.00	0.59	0.00	0
Decision Tree	4349	43	14	0	0.99	0.99	0.44	0.99	0.00	0.44	-0.01	0.0
Naive Bayes	2802	1590	13	**1**	0.78	0.64	0.35	0.99	0.03	0.35	-0.03	0.0006
SMO	**4392**	**0**	14	0	0.99	1.00	0.28	0.99	0.00	0.28	0.00	0.0
KNN	4385	7	14	0	0.99	1.00	0.50	0.99	0.00	0.50	0.00	0.0
Logistic Regression	4373	19	14	0	0.99	0.99	0.36	0.99	0.00	0.36	0.00	0.0
Perceptron	**4392**	**0**	14	0	0.99	1.00	0.20	0.99	0.00	0.20	0.00	0.0
LDA	4287	105	**13**	**1**	0.98	0.97	0.66	0.99	**0.07**	**0.66**	0.02	**0.09**

medida de TP, e de FN e FP. A regressão logística só não possui melhores valores quanto a TP e FN do que o *Naive Bayes*, sendo que este possui um valor de FP muito elevado.

Embora os resultados com o conjunto de dados empiricamente estabelecido indiquem o bom desempenho de alguns classificadores, é essencial avaliar o desempenho com o conjunto de teste. Assim, utilizamos os modelos gerados para a primeira parte do conjunto de dados para testar com a segunda parte do conjunto de dados. Como o classificador não foi treinado com esse segundo conjunto, espera-se, se o vocabulário da instabilidade for comum a projetos escritos em Javascript, um desempenho similar.

No entanto, não foi isso o observado. Ao testar cada classificador com o segundo conjunto de dados, contendo 4.392 casos de testes não instáveis e 15 instáveis, o resultado caí significativamente para a classe *True*. Na Tabela 4 é possível perceber que os resultados da média ponderada são bons por um simples motivo: o desbalanceamento do conjunto de dados. Instâncias de testes comuns são mais simples de serem identificadas do que instâncias de *flaky tests*. Por este motivo vamos focar nos resultados da classe *True*.

O melhor resultado foi para o algoritmo LDA com *recall* de 0.09 e F1-score de 0.07, encontrando apenas 1 dos 14 *flaky tests* existentes no conjunto de teste. Isto revela que o domínio do projeto e dos termos utilizados nos casos de teste afeta a causa raiz de instabilidade e consequentemente nos termos instáveis. Para que um vocabulário instável seja generalizável, é preciso que ele seja treinado com um conjunto completo de projetos de domínios diferentes.

Concluindo, a maioria dos classificadores teve um desempenho muito bom quando os conjuntos de dados de treinamento e teste são os mesmos. Neste cenário, o algoritmo de regressão logística teve o melhor resultado e o Naive Bayes o pior desempenho. Porém, quando o modelo foi testado com um conjunto de dados distinto, os classificadores em geral não tiveram bom desempenho. Isto revela que o vocabulário presente em casos de testes de diferentes contextos podem ser diferentes, e consequentemente a raiz de instabilidades em testes também.

Resposta para QP1: Apresentamos bons resultados durante a execução da maioria dos classificadores utilizando o conjunto de treino e teste, tendo como o melhor resultado o algoritmo de regressão logística. Porém, ao classificar um conjunto de teste distinto do conjunto de treinamento, o desempenho não foi bom: embora a precisão tenha sido razoável e todas as outras medidas (F1-score, AUC e MCC) foram ruins.

5.2 Vocabulário associado à instabilidade

Analisamos na Tabela 5 os 20 primeiros termos com maior ganho de informação para identificar testes instáveis. O ganho de informação foi calculado conforme apresentado por Camara et al. [10], utilizando a função *mutual_info_classif* do SciKit-Learn.

O termo *then* aparece em projetos de diferentes domínios. Isto mostra que algumas palavras do vocabulário podem ser genéricas. Existem palavras específicas no contexto do projeto, por exemplo,

Tabela 5: Termos com maior ganho de informação para *flaky tests*.

Rank	Token	Ganho de informação	# *Flaky tests*
1	then	0.034	41
2	gd	0.024	27
3	function	0.021	32
4	done	0.021	36
5	cy	0.020	24
6	getByTestID	0.018	21
7	click	0.018	31
8	it	0.017	19
9	var	0.016	27
10	0	0.016	44
11	Plotly	0.014	16
12	layout	0.014	17
13	1	0.014	37
14	should	0.011	17
15	await	0.011	33
16	return	0.011	23
17	type	0.011	20
18	data	0.010	19
19	const	0.009	46
20	page	0.009	13

o termo *browser* está presente em *flaky tests* do projeto Next.js e está vinculado com a plataforma de execução do teste.

A presença do termo *then* em primeiro lugar e do termo *await* (ganho de informação de: 0.034) em 15° pode representar problemas com a espera assíncrona. Como a maioria dos projetos selecionamos são relacionados a interface do usuário, esse termo também pode ser utilizado para esperar que uma função modifique algo na tela para então verificar o resultado final do teste. Quando funções assíncronas são executadas e demoram para retornar seu resultado, ocorre o *timeout* no teste. Esse evento não ocorrerá em todas execuções, somente naquelas que demorarem para executar. Isso ocorre na prática e muitos desenvolvedores, por não conseguir prever a instabilidade, adicionam *delays* no código para mitigá-las [43].

Os termos *function*, *return*, *0* e *1* tiveram um bom ganho de informação devido à existência de *fixtures* e à proporção em que aparecem em testes instáveis e não instáveis. O termo *function* é utilizado na criação de *fixtures*, geralmente seguido de um *return*. Os *fixtures* são utilizados para configuração de dados, cenários, funções de *callback*, dentre outros, com o objetivo de construir as interfaces gráficas e mecanismos de interação nos projetos analisados. Quanto aos termos *0* e *1*, eles geralmente são utilizados para verificar o tamanho de elementos e *arrays*, acessar posições de *arrays*, realizar comparações, acessar medidas de tamanho e altura, valores de coordenadas. Isso vai de encontro com valores nos limites de classes de particionamento, o que, neste caso, revela instabilidades.

Encontramos duas palavras associadas à instabilidade que estão relacionadas com a utilização da ferramenta de teste de front-end web Cypress, são elas : *cy* e *getByTestID*. No trabalho apresentado por Pinto et al. [40] aparecem palavras semelhantes, por exemplo: *id* e *getId*. No conjunto de dados que serviu de base para obtenção dos casos de teste instáveis [43], foi identificada uma categoria comum de instabilidade relacionada com interação com o arcabouço de teste (*incorrect test runner interaction*), possivelmente indicando que algumas ferramentas de teste são mais propensas às instabilidades, conforme sugerido por Romano et al. [43] ou que os elementos do software sob teste que estão sendo tratados por algumas funções específicas do arcabouço de teste estão mais sujeitos à instabilidade. Os termos relacionados ao framework que executa os testes, *cy* e *getByTestID*, podem ser explicados pela complexidade dos testes que são executados nessas plataformas. Acreditamos que tais ferramentas para execução de testes facilitam a identificação de *flaky tests* pois permitem a execução de testes de sistemas, que são mais propensos à instabilidade. Testes mais complexos necessitam de alta integração de recursos e podem ser executados em diferentes ordens. No trabalho apresentado por Pinto et al. [40], é utilizada uma característica relacionada ao tamanho do teste (linhas de código) e ela está associada com instabilidade, com alto ganho de informação. Entretanto, a quantidade de linhas não é uma medida relacionado ao vocabulário, mas uma medida de tamanho do método.

Para validar se as palavras instáveis obtidas à partir dos 144 flakies relatados pelos desenvolvedores no GitHub são generalizáveis o suficiente, obtivemos as palavras com maior ganho de informação para os *flaky tests* obtidos pela técnica de reexecução nos projetos Next.js e Moleculer. Desta forma podemos verificar se existe uma intersecção com os termos da Tabela 5.

Para o projeto Moleculer, as palavras com maior ganho de informação estão relacionadas a serialização de objetos (*PACKET_EVENT*, *obj*, *P*, *serialize*). Para o projeto Next.js, a palavra com maior ganho de informação para identificar *flaky tests* é *await*, revelando que assincronicidade é uma causa de instabilidade que pode ser descoberta à partir do vocabulário. A segunda palavra com maior ganho de informação neste conjunto é *browser*, presente em 6 *flaky tests*. Isto sugere que a causa raiz de instabilidade relacionada a 'plataforma de execução' se faz presente neste novo conjunto de *flaky tests*.

Resposta para QP2: O vocabulário relacionado com a instabilidade possui palavras relacionadas com espera assíncrona, como: *then*, *await*, *return*. Existem palavras relacionadas à visualização, interação com usuário, por exemplo: *layout*, *gd*, *plot* e *click*. Encontramos duas palavras associadas à utilização do arcabouço Cypress: *cy* e *getByTestID*. Também observam-se palavras relacionadas com *fixtures*, tais como: *function* e *return*

6 DISCUSSÕES

Este trabalho seguiu a implementação dos mesmos 5 modelos de classificação utilizados por Pinto et al. [40] e incrementados por Camara et al. [10]. Enquanto no estudo de Pinto et al. [40] o Random Forest obteve o melhor resultado, nos estudos de Camara et al. [10] e neste trabalho a Regressão Logística foram melhores. Entretanto, em todos os estudos os melhores classificadores apresentaram bom desempenho. Em relação ao algoritmo com pior resultado, este estudo e o de Pinto et al. [40] tiveram resultados concordantes, embora a revocação, F1 Score, MMC e AUC observadas neste trabalho tenham obtido resultado inferiores àquele.

As palavras mais comuns no vocabulário original [40] são: *job*, *table* e *action*. No vocabulário proposto por Camara et al. [10] são: *job*, *table* e *service*. Tal semelhança ocorre pelo fato de utilizarem conjuntos de dados semelhantes, sobre casos de testes de aplicações Java. O vocabulário instável obtido neste trabalho é apresentado na Tabela 5. Nele, as 4 palavras com maior ganho de informação são: *then gd done* e *function*. Como se tratam de linguagem de programação distintas, essa diferença era esperada. Ao mesmo tempo, observando as demais palavras dos vocabulários, é possível detectar alguns padrões semelhantes.

O termo que possui maior relevância em termos de ganho de informação é o *then* que está fortemente associado a instabilidades ocasionadas pela espera assíncrona. Duas palavras presentes no vocabulário (entre projetos) de Camara et al. [10] que também estão presentes em nosso vocabulário são: *await* e *return*. Este fato sugere que a espera assíncrona é um problema existente em diversos contextos de casos de testes.

Assim, observa-se uma coincidência de conceitos relacionados a terminologia adotada nos dados analisados, apesar da diferença de linguagem adotada (Java e Javascript) e domínio das aplicações. Embora o estudo de Camara et al. [10] apresente indícios de que os modelos treinados para um conjunto de projetos não possua desempenho adequada para outros projetos de software na mesma linguagem, podemos observar que existe uma interseção nos vocabulários observados.

7 AMEAÇAS À VALIDADE

Uma ameaça à validade interna é a representatividade dos dados dentro do domínio. Por exemplo, para se obter dados de aplicações típicas Javascript é necessário selecionar projetos relevantes. Para tratar essa ameaça foram selecionados projetos considerados relevantes pelo número de estrelas. Utilizamos projetos com alta relevância no GitHub, por exemplo: o 'influxdata/influxdb' que possui 23 mil estrelas e o 'plotly/plotly.js' que possui 14 mil estrelas, entre outros. Num total de 32 projetos selecionados obtivemos um conjunto de dados com somente 144 instâncias sobre casos de testes instáveis que foram revisados manualmente à partir de *commits*. O número baixo de dados sobre casos de testes instáveis pode ser uma ameaça à validade interna. Além disso, é possível que alguns dos casos de teste que consideramos como não instáveis sejam, na verdade, instáveis. Estes problemas estão relacionados com o custo alto para identificação de *flaky tests* em abordagens convencionais. Sugerimos a aplicação de abordagens de reexecução para todos os projetos e com uma quantidade elevada de repetições, e um conjunto de projetos maior para construir uma base de dados mais robusta para mitigar esta ameaça.

O desbalanceamento da base de dados construída neste trabalho em relação as classes *flaky* e não *flaky* é uma ameaça a validade que afeta os resultados dos modelos de classificação. Este problema está relacionado com a quantidade de tempo e recurso necessário para identificação de *flaky tests* em Javascript. Neste trabalho, foi utilizada a mesma abordagem de trabalho anterior [40], reduzindo-se a quantidade de instâncias de casos de teste não *flaky*. No entanto, outras estratégias de balanceamento podem ser investigadas, como, por exemplo, inserindo instâncias da classe *flaky* para validar os resultados. Utilizar a validação cruzada com k-partições poderia melhorar os resultados, porém optamos por ser fiel aos trabalhos anteriores, mas este ponto também pode ser reavaliado em novos estudos.

Para os projetos analisados, foi considerado que os casos de teste foram implementados na mesma linguagem utilizada para a parte da aplicação sob teste. De forma geral, isso se mantém quando consideram-se testes de unidade e de integração, que são aqueles geralmente encontrados em projetos de software. Entretanto, seria possível ter casos de testes escritos em outras linguagens, distintas daquela utilizada para a parte sob teste da aplicação. Por exemplo, em testes de sistema seria possível utilizar scripts com comandos do interpretador do sistema (Bash) ou de configuração de containers de software (Docker). Embora esses tipos de teste não sejam contemplados por esse estudo, cabe destacar que, para os projetos analisados, não foram encontrados testes com tais características.

Uma ameaça a validade externa é a limitação do escopo do conjunto de dados utilizado: o modelo pode se limitar ao domínio Javascript para interfaces web. Dessa forma, devem ser considerados projetos de diversos domínios e características para constituir o conjunto de dados para criação dos modelos de classificação.

8 CONCLUSÕES

Este estudo contemplou o estabelecimento de um conjunto de dados para avaliação de modelos de classificação de casos de teste quanto à instabilidade e a criação desses modelos de classificação para identificar *flaky tests*.

A primeira contribuição foi um conjunto de dados com o código-fonte de casos de teste instáveis e não instáveis em Javascript, construído a partir de estudos empíricos e da aplicação da técnica de reexecução. Diante das limitações observadas na Seção 7, em trabalhos futuros o conjunto de dados será expandido, incluindo mais projetos e os avaliando quanto a *flaky tests* com técnicas de reexecução, permitindo a execução de estudos mais robustos com algoritmos supervisionados de aprendizagem de máquina.

A segunda contribuição foram os modelos para classificação construídos com aprendizado de máquina para prever *flaky tests* à partir do código-fonte. Considerando modelos construídos e testados com o mesmo conjunto de projetos, foi possível prever instabilidades em casos de testes Javascript utilizando vocabulário instável com uma precisão considerável. Esta abordagem é vantajosa e promissora pois economiza tempo necessário para identificação de instabilidades. No entanto, ao utilizar os classificadores com projetos distintos, ainda que em Javascript, o resultado fica aquém do desejável, ficando evidente a necessidade de aprimorar a construção de modelos de classificação baseados em vocabulários.

O vocabulário pode acelerar a identificação da causa raiz da falha intermitente. Por exemplo, em *flaky tests* de espera assíncrona sabemos que os temos *then*, *await*, *return* e *done* podem estar relacionados diretamente com a instabilidade. Conhecendo o vocabulário instável em casos de teste Javascript facilitamos a identificação e possível correção de instabilidades.

A predição de instabilidade ainda não é capaz de substituir a reexecução como abordagem principal para identificação de *flaky tests* por dois motivos: limitações referentes ao conjunto de dados utilizado e baixa generalização dos termos instáveis para diferentes contextos. Porém, atualmente apresenta resultados bons como técnica complementar à reexecução, com intuito de reduzir o custo da identificação de instabilidades em testes.

AGRADECIMENTOS

O presente trabalho foi realizado com apoio da Coordenação de Aperfeiçoamento de Pessoal de Nível Superior – Brasil (CAPES) – Código de Financiamento 001.

REFERÊNCIAS

[1] 2018. *International Conference on Software Engineering* (Gothenburg, Suécia).
[2] 2018. *International Working Conference on Source Code Analysis and Manipulation* (Madrid, Espanha).
[3] 2021. *International Conference on Software Engineering* (Madrid, Espanha).
[4] Abdulrahman Alshammari, Christopher Morris, Michael Hilton, and Jonathan Bell. 2021. FlakeFlagger: Predicting Flakiness Without Rerunning Tests, See [3], 1572–1584.
[5] Gabriele Bavota, Abdallah Qusef, Rocco Oliveto, Andrea De Lucia, and David Binkley. 2012. An empirical analysis of the distribution of unit test smells and their impact on software maintenance. In *28th IEEE International Conference on Software Maintenance* (Trento,Italy). 56–65.
[6] Jonathan Bell, Owolabi Legunsen, Michael Hilton, Lamyaa Eloussi, Tifany Yung, and Darko Marinov. 2018. DeFlaker: Automatically Detecting Flaky Tests, See [1], 433–444.
[7] Jonathan Bell, Owolabi Legunsen, Michael Hilton, Lamyaa Eloussi, Tifany Yung, and Darko Marinov. 2018. DeFlaker: Automatically Detecting Flaky Tests, See [1], 433–444.
[8] Antonia Bertolino, Emilio Cruciani, Breno Alexandro Ferreira de Miranda, and Roberto Verdecchia. 2020. *Know your neighbor: fast static prediction of test flakiness.* techreport ISTI-2020-TR/001. Istituto di Scienza e Tecnologie dell'Informazione "A. Faedo", Pisa, Itália.
[9] Bruno Camara, Marco Silva, Andre Endo, and Silvia Vergilio. 2021. On the Use of Test Smells for Prediction of Flaky Tests. In *VI Brazilian Symposium on Systematic and Automated Software Testing (SAST'21)* (6 ed.) (Joinville, SC, Brasil). 46–54.
[10] Bruno Henrique Pachulski Camara, Marco Aurélio Graciotto Silva, Andre Takeshi Endo, and Silvia Regina Vergilio. 2021. What is the Vocabulary of Flaky Tests? An Extended Replication. In *29th IEEE/ACM International Conference on Program Comprehension (ICPC 2021)* (Madrid, Espanha). 444–454.
[11] David Cournapeau et al. 2007. scikit-learn. Programa de computador. https://scikit-learn.org/
[12] Cypress. 2017. Cypress. Programa de computador. https://www.cypress.io/
[13] Arie Van Deursen, Leon Moonen, Alex Bergh, and Gerard Kok. 2001. Refactoring Test Code. In *2nd International Conference on Extreme Programming and Flexible Processes in Software Engineering (XP 2001)* (Villasimius, Sardinia, Itália). 92–95.
[14] Moritz Eck, Fabio Palomba, Marco Castelluccio, and Alberto Bacchelli. 2019. Understanding Flaky Tests: The Developer's Perspective. In *2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Tallinn, Estônia). 830–840.
[15] Lamyaa Eloussi. 2015. *Determining Flaky Tests from Test Failures.* mathesis. University of Illinois at Urbana-Champaign, Urbana, Illinois,. Advisor(s) Darko Marinov. http://hdl.handle.net/2142/78543
[16] Facebook. 2014. Jest. Programa de computador. https://jestjs.io/
[17] Usama Fayyad, Gregory Piatetsky-Shapiro, and Padhraic Smyth. 1996. From Data Mining to Knowledge Discovery in Databases. *AI Magazine* 17, 3 (Sept.–Nov. 1996), 37–54.
[18] Martin Gruber, Stephan Lukasczyk, Florian Kroiß, and Gordon Fraser. 2021. An Empirical Study of Flaky Tests in Python. In *2021 14th IEEE International Conference on Software Testing, Verification and Validation* (Porto de Galinhas, PE, EUA). 148–158.
[19] Guillaume Haben, Sarra Habchi, Mike Papadakis, Maxime Cordy, and Yves Le Traon. 2021. A Replication Study on the Usability of Code Vocabulary in Predicting Flaky Tests. In *2021 Mining Software Repositories Conference* (18 ed.) (Madrid, Espanha). 219–229.
[20] Mark Harman and Peter O'Hearn. 2018. From Start-ups to Scale-ups: Opportunities and Open Problems for Static and Dynamic Program Analysis, See [2], 1–23.
[21] Mark Harman and Peter O'Hearn. 2018. From Start-ups to Scale-ups: Opportunities and Open Problems for Static and Dynamic Program Analysis, See [2], 1–23.
[22] Negar Hashemi, Amjed Tahir†, and Shawn Rasheed. 2022. An empirical study of flaky tests in Javascript. In *38th IEEE International Conference on Software Maintenance and Evolution* (38 ed.) (Limassol, Chipre). 24–34.
[23] James Henry et al. 2019. TypeScript-ESLint. Programa de computador. https://github.com/typescript-eslint/typescript-eslint
[24] Kim Herzig and Nachiappan Nagappan. 2015. Empirically Detecting False Test Alarms Using Association Rules. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering* (37 ed.) (Florence, Itália). 39–48.
[25] Vojtech Jina et al. 2012. Karma. Programa de computador. https://karma-runner.github.io
[26] Tariq M. King, Dionny Santiago, Justin Phillips, and Peter J. Clarke. 2018. Towards a Bayesian Network Model for Predicting Flaky Automated Tests. In *2018 IEEE International Conference on Software Quality, Reliability and Security Companion (QRS-C)* (Lisbon, Portugal). 100–107.
[27] Wing Lam, Patrice Godefroid, Suman Nath, Anirudh Santhiar, and Suresh Thummalapenta. 2019. Root causing flaky tests in a large-scale industrial setting. In *28th ACM SIGSOFT International Symposium on Software Testing and Analysis.* 101–111.
[28] Wing Lam, Patrice Godefroid, Suman Nath, Anirudh Santhiar, and Suresh Thummalapenta. 2019. Root Causing Flaky Tests in a Large-Scale Industrial Setting. In *28th ACM SIGSOFT International Symposium on Software Testing and Analysis* (Beijing, China). 101–111.
[29] Wing Lam, Kıvanç Muşlu, Hitesh Sajnani, and Suresh Thummalapenta. 2020. A Study on the Lifecycle of Flaky Tests. In *42nd International Conference on Software Engineering* (Seoul, Coréia do Sul). 1471–1482.
[30] Wing Lam, Reed Oei, August Shi, Darko Marinov, and Tao Xie. 2019. iDFlakies: A Framework for Detecting and Partially Classifying Flaky Tests. In *2019 12th IEEE Conference on Software Testing, Validation and Verification (ICST)* (Xi'an, China). 312–322.
[31] Jeff Listfield. 2017. Where do our flaky tests come from? Página Web. https://testing.googleblog.com/2017/04/where-do-our-flaky-tests-come-from.html
[32] Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. 2014. An Empirical Analysis of Flaky Tests. In *22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering* (Hong Kong, China). 643–653.
[33] Meta. 2016. Yarn. Programa de computador. https://yarnpkg.com/
[34] John Micco. 2016. Flaky tests at Google and how we mitigate them. *Online] https://testing. googleblog. com/2016/05/flaky-tests-at-google-and-how-w e. html* (2016).
[35] John Micco. 2017. The State of Continuous Integration Testing@ Google.(2017).
[36] Charles Miranda, Guilherme Avelino, Pedro Santos Neto, and Victor da Silva. 2021. Uma Análise da Co-Evolução de Teste em Projetos de Software no GitHub. In *IX Workshop de Visualização, Evolução e Manutenção de Software (VEM 2021)* (12 ed.) (Joinville, SC, Brasil). 36–40.
[37] Jesús Morán, Cristian Augusto, Antonia Bertolino, Claudio De La Riva, and Javier Tuya. 2020. FlakyLoc: Flakiness Localization for Reliable Test Suites in Web Applications. *Journal of Web Engineering* 19, 2 (June 2020), 267–296.
[38] NPM. 2010. npm. Programa de computador. https://www.npmjs.com/
[39] Jason Palmer. 2019. Test Flakiness – Methods for identifying and dealing with flaky tests. https://engineering.atspotify.com/2019/11/18/test-flakiness-methods-for-identifying-and-dealing-with-flaky-tests/
[40] Gustavo Pinto, Breno Miranda, Supun Dissanayake, Marcelo d'Amorim, Christoph Treude, and Antonia Bertolino. 2020. What is the Vocabulary of Flaky Tests?. In *17th International Conference on Mining Software Repositories (MSR)* (17 ed.) (Seoul, Coréia do Sul). 492–502.
[41] Tom Preston-Werner, Chris Wanstrath, P. J. Hyett, and Scott Chacon. 2008. GitHub. Programa de computador. https://github.com
[42] Solange Oliveira Rezende. 2005. *Sistemas inteligentes: fundamentos e aplicações* (1 ed.). Barueri, SP, Brasil. 550 pages.
[43] Alan Romano, Zihe Song, Sampath Grandhi, Wei Yang, and Weihang Wang. 2021. An Empirical Analysis of UI-based Flaky Tests, See [3], 1585–1597.
[44] Swapna Thorve, Chandani Sreshtha, and Na Meng. 2018. An Empirical Study of Flaky Tests in Android Apps. In *2018 IEEE International Conference on Software Maintenance and Evolution (ICSME)* (Madrid, Espanha). 534–538.
[45] Michele Tufano, Fabio Palomba, Gabriele Bavota, Massimiliano Di Penta, Rocco Oliveto, Andrea De Lucia, and Denys Poshyvanyk. 2016. An Empirical Investigation into the Nature of Test Smells. In *31st IEEE/ACM International Conference on Automated Software Engineering* (Singapura). 4–15.
[46] Roberto Verdecchia, Emilio Cruciani, Breno Miranda, and Antonia Bertolino. 2021. Know You Neighbor: Fast Static Prediction of Test Flakiness. *IEEE Access* (2021).